



AZ-204

Day 4: Messaging & Event-Driven Architecture on Azure

Master asynchronous communication patterns, Azure messaging services, and event-driven design — the backbone of modern distributed systems.

AZ-204 TRAINING

DAY 4 OF 5

Day 4 Agenda

01

Messaging Fundamentals

Why async communication matters in distributed systems

03

Service Bus Deep Dive

Queues, topics, subscriptions, sessions, dead-letter queues

02

Azure Messaging Services Overview

Queue Storage, Service Bus, Event Grid, Event Hubs

04

Event-Driven Services

Event Grid vs. Event Hubs — patterns and real-world scenarios



Why Asynchronous Messaging?

Synchronous calls between services create tight coupling — one slow or failed service can cascade failures across the system. Async messaging decouples producers from consumers, enabling resilient, scalable distributed architectures.

Core Messaging Concepts

Producer / Consumer

The producer sends messages; the consumer receives and processes them independently — no direct dependency between the two.

Message Queue

A buffer that stores messages until the consumer is ready. Guarantees at-least-once delivery and load leveling.

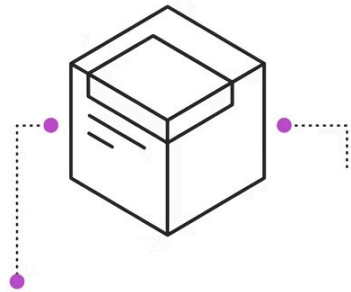
Publish / Subscribe

One producer publishes a message; multiple independent consumers receive their own copy via topic subscriptions.

Dead-Letter Queue

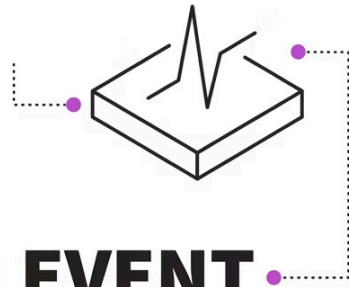
A holding area for messages that cannot be processed — enables diagnostics and manual re-processing without data loss.

Message vs. Event — Know the Difference



MESSAGE

**carries raw data,
producer
expects
consumer to act,
ownership
transfers, used in
Service Bus and
Queue Storage**



EVENT

**lightweight
notification,
producer does
not care who acts,
no ownership
transfer, used in
Event Grid and
Event Hubs**

Why It Matters for AZ-204

Choosing the wrong abstraction is a common exam trap. Ask: **Does the sender care what happens to the data?**

- **Yes** → Use a message broker (Service Bus)
- **No** → Use an event service (Event Grid / Event Hubs)

Azure Messaging Services — The Big Picture

Azure provides four primary services for asynchronous communication. Each is purpose-built for a specific pattern and scale requirement.



Azure Queue Storage

Simple, durable message queuing at massive scale



Azure Service Bus

Enterprise message broker with advanced features



Azure Event Grid

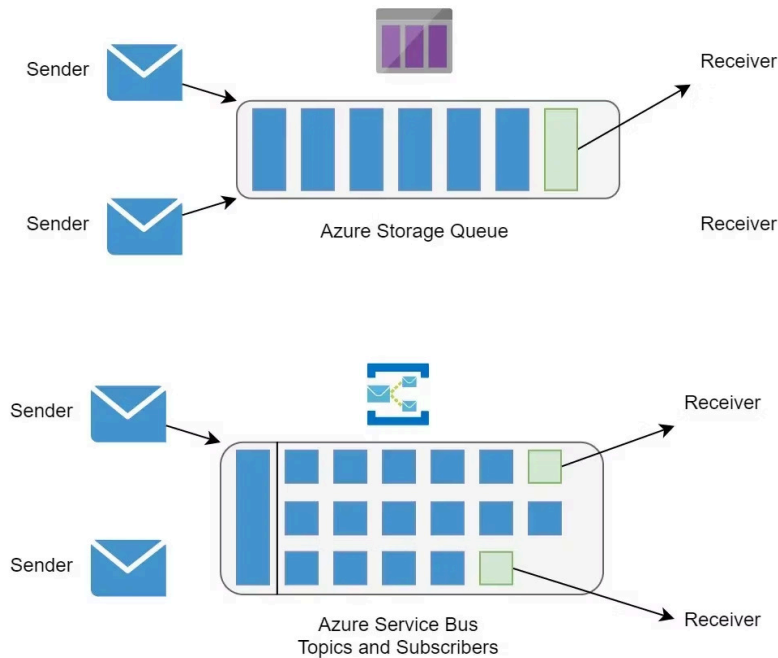
Reactive event routing at serverless scale



Azure Event Hubs

High-throughput telemetry and event streaming

Azure Queue Storage — Overview



When to Use It

- Need a simple, cheap queue — no advanced routing
- Queue must store **more than 80 GB** of messages
- Want audit logs via Azure Storage analytics

Key Characteristics

- Messages up to **64 KB** each
- Retention up to **7 days**
- REST-based API, SDK support in all major languages
- Part of the Azure Storage account — no separate namespace

Queue Storage vs. Service Bus

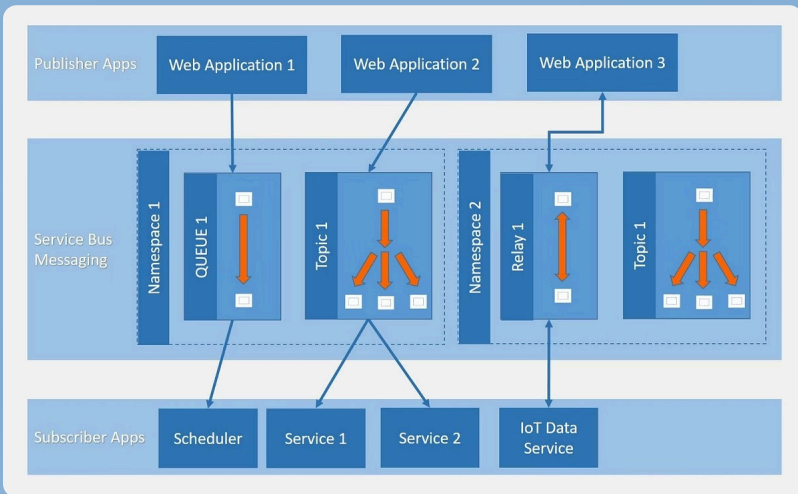
Feature	Queue Storage	Service Bus
Max message size	64 KB	256 KB (Standard) / 100 MB (Premium)
Max queue size	Unlimited (storage limit)	1 GB – 80 GB
Message TTL	Up to 7 days	Up to 14 days (configurable)
Ordering guarantee	Best effort (FIFO not guaranteed)	FIFO with message sessions
Dead-letter queue	Not built-in	Built-in
Pub/Sub (Topics)	No	Yes
Duplicate detection	No	Yes
Transactions	No	Yes

 **Exam tip:** If the scenario mentions ordered delivery, pub/sub, transactions, or duplicate detection — the answer is Service Bus.

CHAPTER 2

Azure Service Bus — Enterprise Messaging

A fully managed enterprise message broker with message queues and publish-subscribe topics. Built for mission-critical workflows that require **ordering**, **transactions**, and **reliable delivery**.



Service Bus: Tiers & Namespaces

Namespace

A namespace is the top-level scoping container for all messaging resources. It has a unique FQDN and hosts your queues and topics.

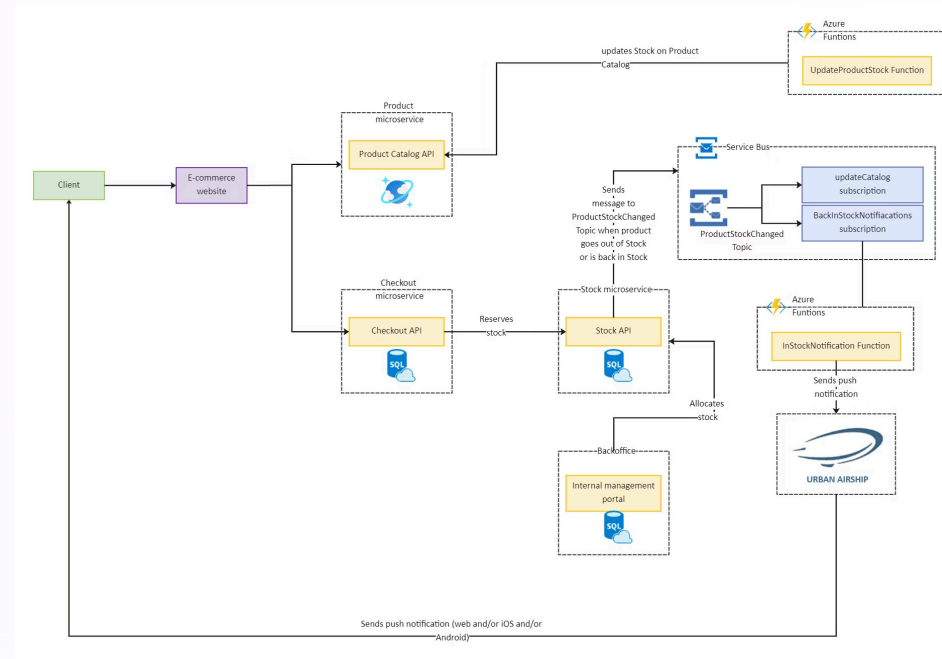
Two Pricing Tiers

Standard

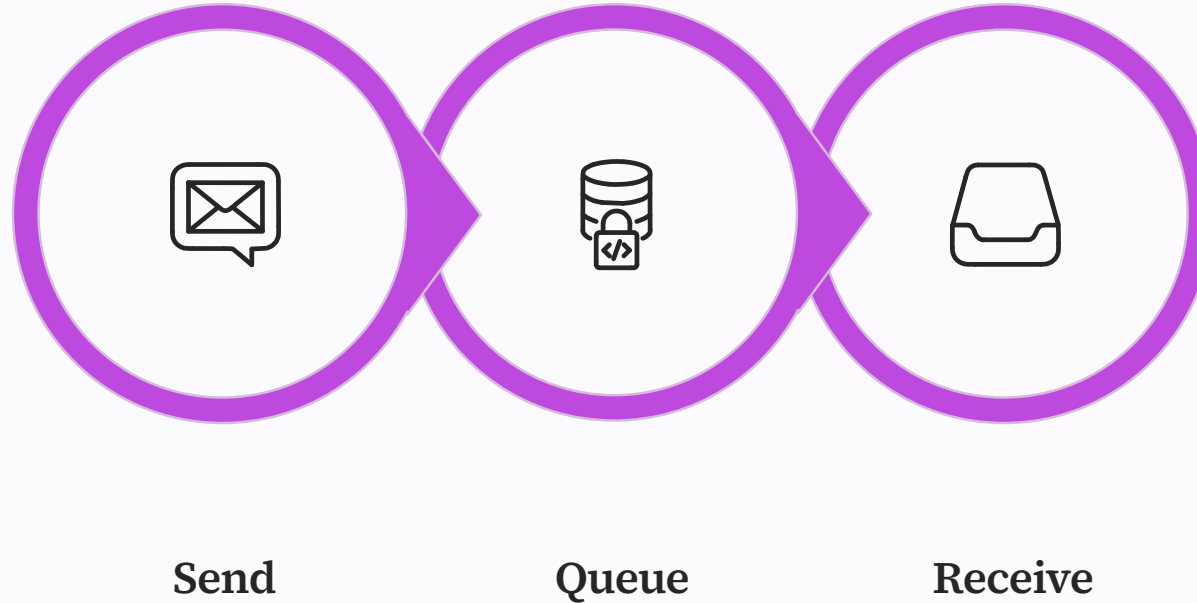
- Shared capacity
- Pay-per-operation
- 256 KB max message

Premium

- Dedicated capacity
- Up to 100 MB messages
- Zone redundancy, VNet



Service Bus Queues



Messages are stored durably and delivered to **one consumer only** (competing consumers pattern). A message is locked during processing — if the consumer crashes, the lock expires and the message becomes available again. This guarantees at-least-once delivery.

Service Bus: Message Lifecycle

1

Active

Message is available in the queue for delivery

2

Locked (In-Flight)

Consumer has received the message; processing is underway

3

Completed

Consumer calls `CompleteAsync()`; message is removed

4

Dead-Lettered

Max delivery count exceeded or explicitly rejected

Dead-Letter Queue (DLQ)

What It Is

Every Service Bus queue and topic subscription gets a built-in sub-queue called the **dead-letter queue**. Messages are moved here automatically when they cannot be delivered.

Common Causes

- Max delivery count exceeded (default: 10)
- Message TTL expired
- Consumer explicitly dead-letters a bad message
- Filter evaluation errors on subscriptions

Real-World Use

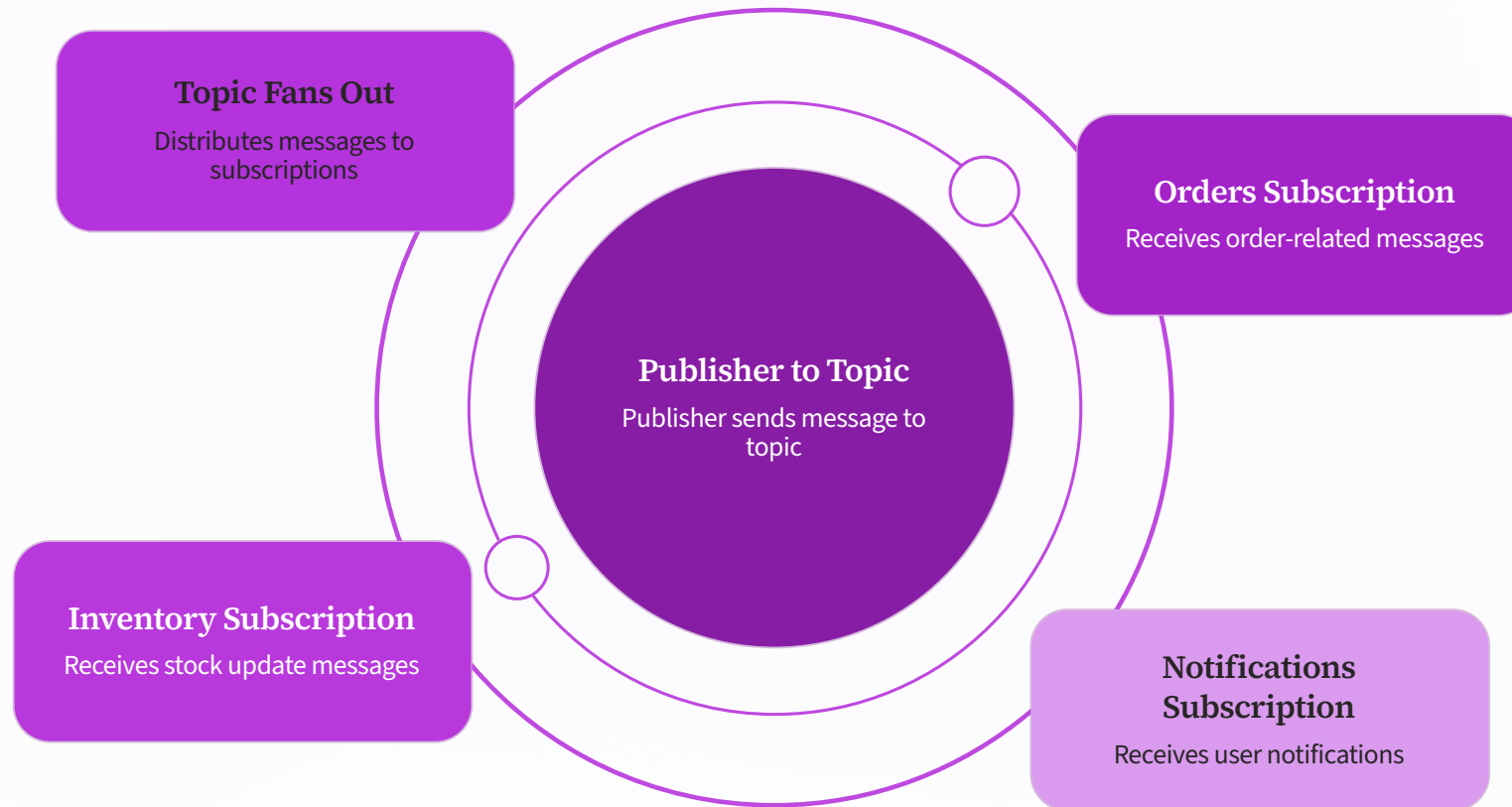
In an e-commerce system, a malformed order message that keeps failing deserialization gets dead-lettered. Ops team inspects it, fixes the bug, and re-submits — without losing the original order.



Exam tip: DLQ path format:

```
queueName/$DeadLetterQueue
```

Topics & Subscriptions — Pub/Sub Pattern



A topic acts like a queue but **fans messages out** to multiple subscriptions. Each subscription gets an independent copy and can apply **SQL-like filters** to receive only relevant messages — ideal for decoupled microservices.

Subscription Filters & Actions

Three Filter Types

Boolean

TrueFilter / FalseFilter —
all or no messages

SQL Filter

`color = 'red' AND`
`quantity > 5` — filter on
properties

Correlation Filter

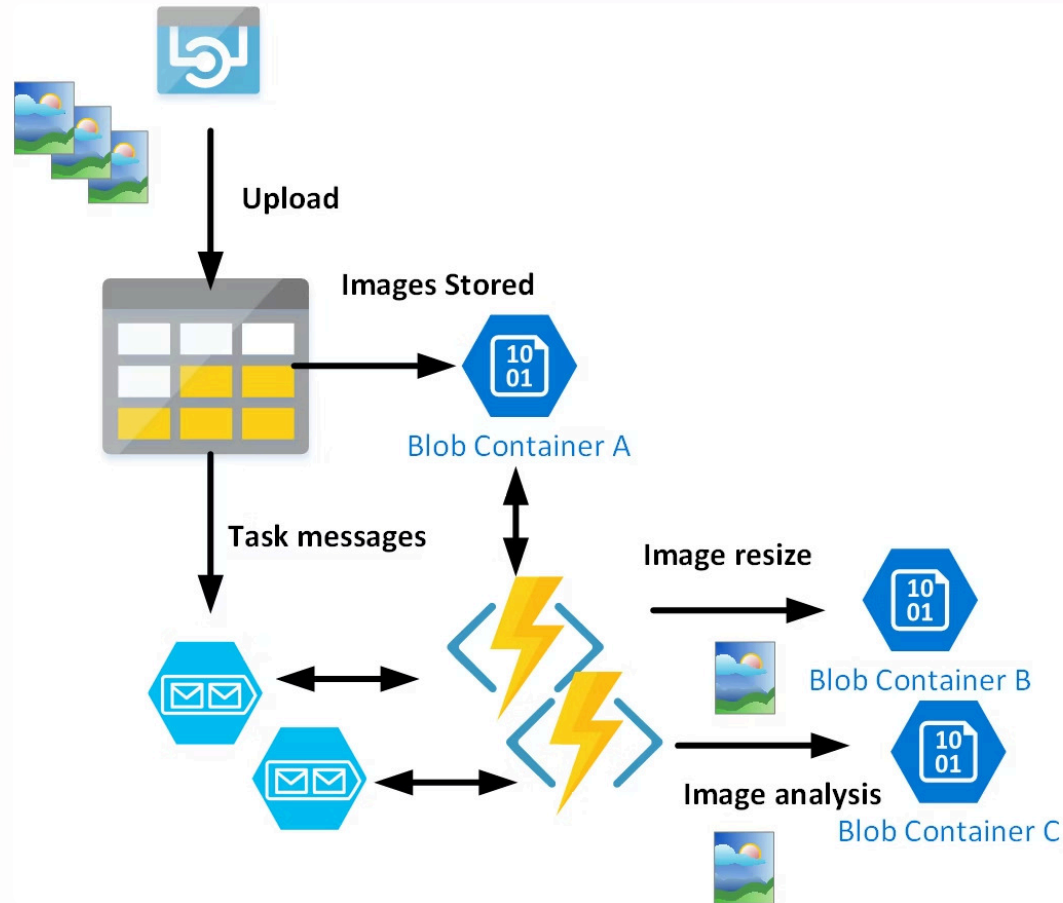
Matches on standard message properties like
`CorrelationId`

Real-World Scenario

An e-commerce platform publishes all orders to a single topic:

- **Inventory service** subscribes with filter `OrderType = 'physical'`
- **Digital fulfillment** subscribes with `OrderType = 'digital'`
- **Analytics service** receives all messages (TrueFilter)

Message Sessions – Guaranteed FIFO



What Are Sessions?

Sessions group related messages under a shared `SessionId`, guaranteeing strict FIFO ordering within the group. Only one consumer processes a session at a time.

When to Use

- Multi-step order workflows (place → pay → ship)
- State machines where step N depends on step N-1
- Per-customer or per-tenant isolation

❏ Sessions must be enabled on the queue or subscription at creation time — it cannot be changed later.

Real-World: E-Commerce Order Processing

E-commerce Order Fulfillment Process



→ Customer places order

Web API publishes order message to Service Bus **topic**

→ Fan-out to services

Inventory, payment, and notification services each receive the message via their own **subscription**

→ Resilient processing

If payment service is down, its messages queue up safely — no lost orders, no cascading failures

CHAPTER 3

Azure Event Grid — Reactive Event Routing

Event Grid is a fully managed event routing service built on a pub/sub model. It routes **discrete events** from sources to handlers with near-real-time latency and serverless scale — **no polling required**.

Event Grid: Key Concepts



Event Source

Where the event originates. Examples: Azure Blob Storage, Resource Groups, your custom app via a custom topic.



Topic

An endpoint where sources send events. **System topics** are built into Azure services; **custom topics** are user-defined.



Event Subscription

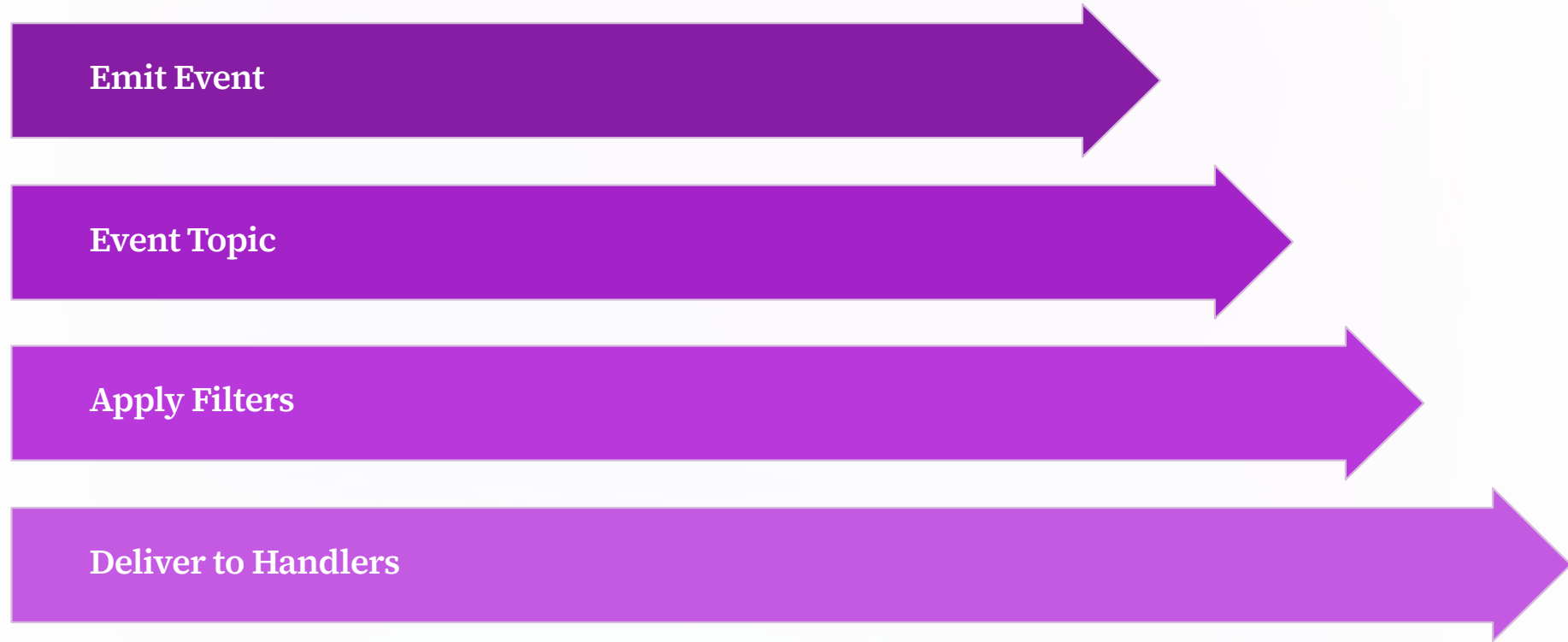
Defines where to route matching events. Supports subject and event-type filters to reduce noise.



Event Handler

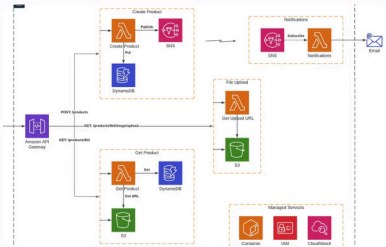
The destination that receives and reacts to events: Azure Functions, Logic Apps, Event Hubs, Webhooks, or Service Bus.

Event Grid Architecture



Event Grid uses a **push delivery model** — handlers don't poll; Event Grid pushes events and retries with exponential back-off for up to 24 hours. This eliminates wasteful polling and dramatically reduces latency.

Event Grid: Real-World Use Cases

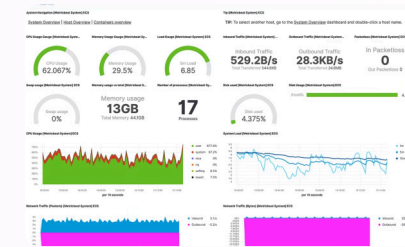


Media Processing

Blob upload event triggers an Azure Function to auto-generate thumbnails and extract metadata.

Infrastructure Automation

Resource Group events trigger compliance checks — flag or auto-remediate non-compliant resources.



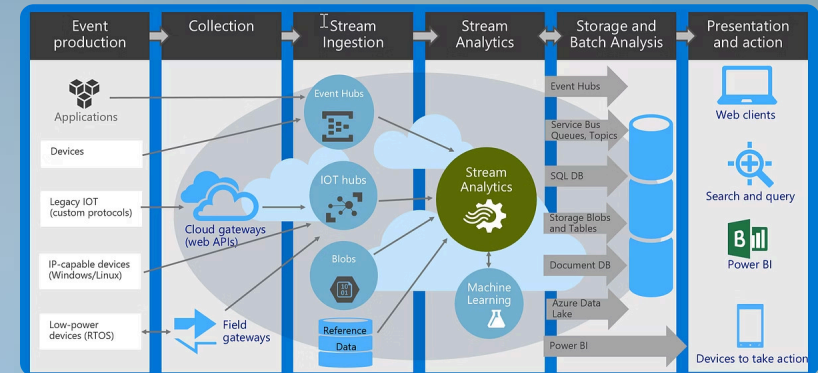
Microservice Coordination

Custom events connect microservices reactively — order confirmed event triggers warehouse and email services simultaneously.

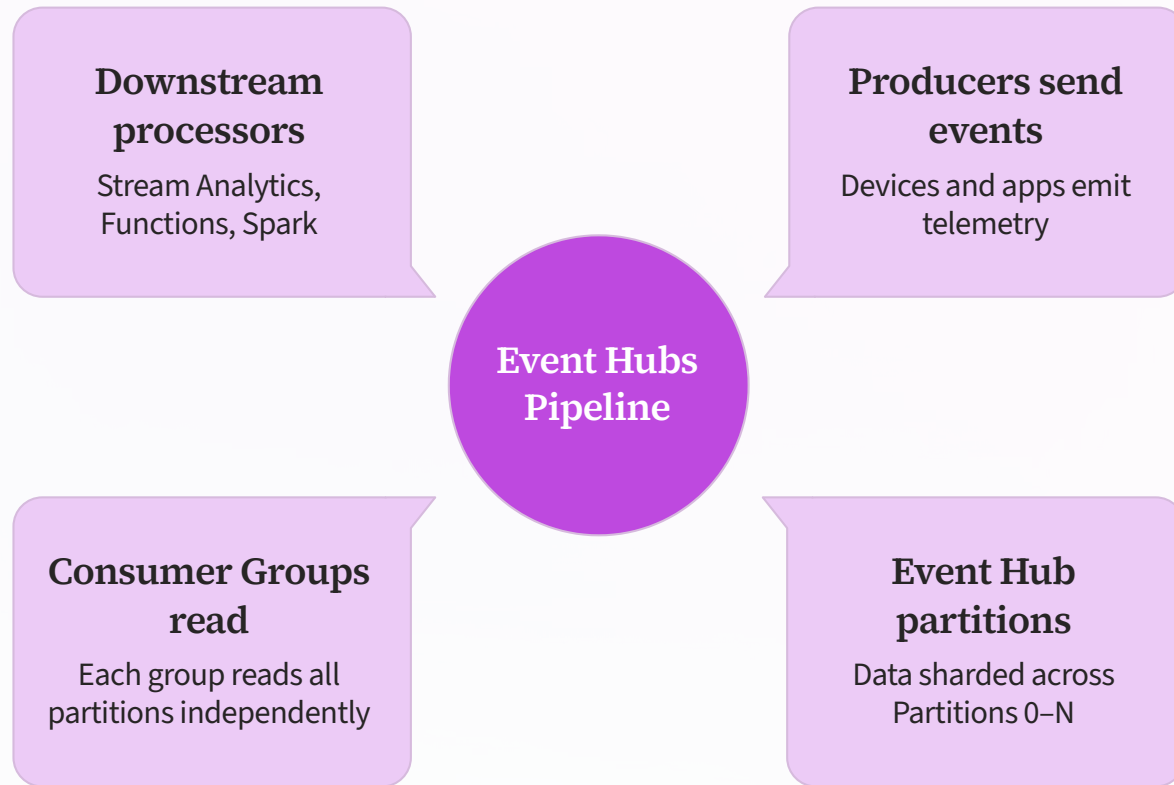
CHAPTER 4

Azure Event Hubs — High-Throughput Event Streaming

Event Hubs is a **big data streaming platform** and event ingestion service capable of receiving and processing **millions of events per second**. Think of it as a distributed log, not a message broker.



Event Hubs: Core Architecture



Partitions are the key to parallelism — each consumer in a consumer group owns one or more partitions, allowing horizontal scale-out. Events are retained for 1–7 days (up to 90 days with Premium/Dedicated).

Event Hubs: Key Concepts

Partitions

Ordered sequence of events within a hub. Throughput scales linearly by adding partitions. Partition count is fixed at creation.

Consumer Groups

A view of the entire event hub. Multiple independent applications each get their own consumer group — they read without interfering with each other.

Capture

Automatically archive streaming events to Azure Blob Storage or Data Lake in Avro format — zero code required.

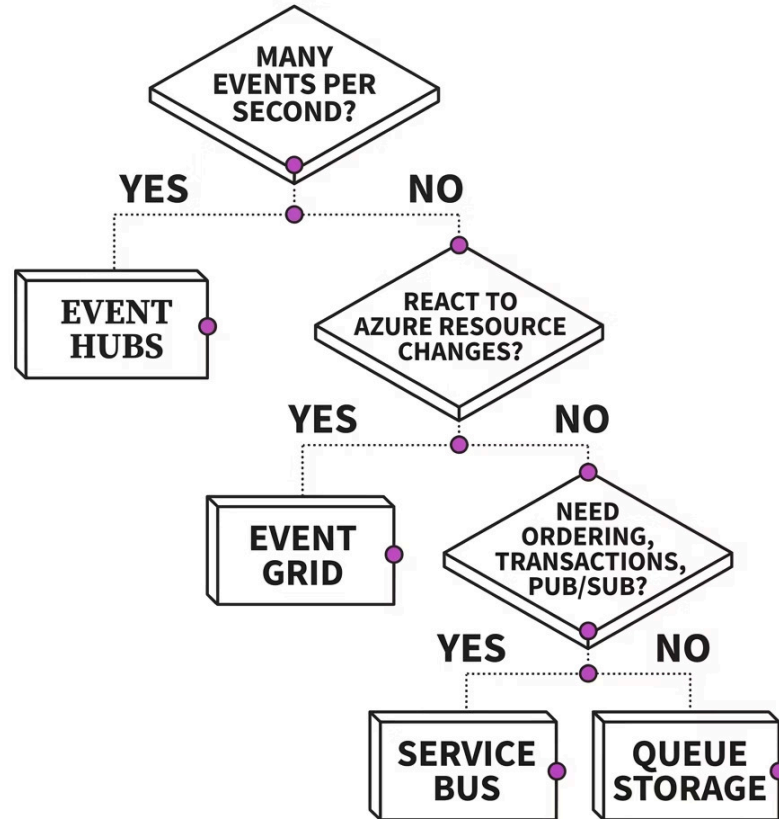
Throughput Units

Each TU = 1 MB/s ingress, 2 MB/s egress. Auto-inflate scales TUs automatically to meet demand in Standard tier.

Event Grid vs. Event Hubs vs. Service Bus

Dimension	Event Grid	Event Hubs	Service Bus
Pattern	Reactive event routing	Event streaming / ingestion	Enterprise message broker
Protocol	HTTP / Webhook	AMQP, HTTPS, Kafka	AMQP, HTTPS
Message size	1 MB	1 MB	256 KB / 100 MB
Throughput	10M events/sec	Millions/sec	Thousands/sec
Retention	24 hrs (retry)	1–90 days	Up to 14 days
Ordering	No	Per partition	FIFO (sessions)
Use case	React to state changes	Telemetry, analytics	Workflow, transactions

Choosing the Right Service



Quick Decision Rules

- **IoT telemetry / clickstreams** → Event Hubs
- **Blob uploaded, resource changed** → Event Grid
- **Order workflow, payment saga** → Service Bus
- **Simple task queue, 64 KB messages** → Queue Storage

📌 On the exam: identify the key requirement first — scale, ordering, or routing — then match to the service.

Real-World: Async Microservices Communication

Command Messages

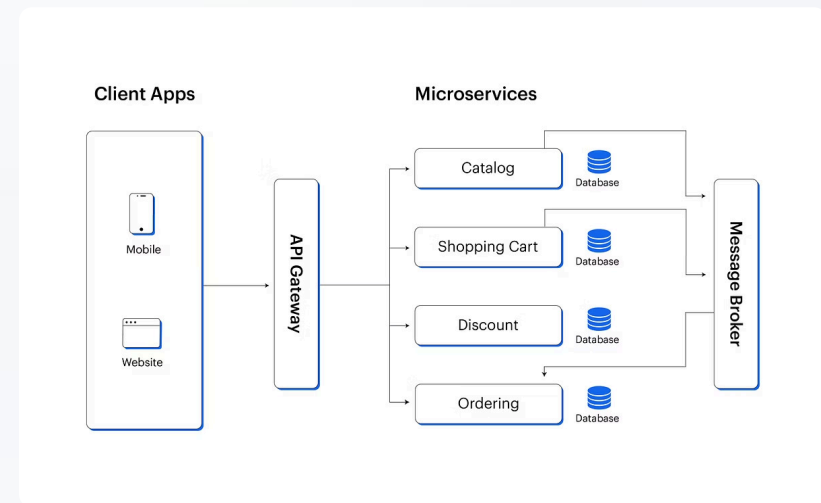
Service A tells Service B to *do something*. Use a Service Bus queue — guaranteed delivery, one consumer.

Domain Events

Something happened — broadcast to all interested services. Use a Service Bus topic with subscriptions.

Infrastructure Events

Azure resource state changes. Use Event Grid to react and trigger automation without polling.



Service Bus: SDK Patterns (C#)

Sending a Message

```
var client = new ServiceBusClient(connStr);  
var sender = client.CreateSender("orders");  
  
var message = new ServiceBusMessage(  
    Encoding.UTF8.GetBytes(orderJson));  
message.SessionId = customerId;  
  
await sender.SendMessageAsync(message);
```

Receiving a Message

```
var processor = client.CreateProcessor(  
    "orders", new ServiceBusProcessorOptions());  
  
processor.ProcessMessageAsync += async args => {  
    var body = args.Message.Body.ToString();  
    await ProcessOrder(body);  
    await args.CompleteMessageAsync(args.Message);  
};  
  
await processor.StartProcessingAsync();
```

Event Grid: Publishing Custom Events (C#)

```
var endpoint = new Uri(topicEndpoint);
var credential = new AzureKeyCredential(key);
var client = new EventGridPublisherClient(
    endpoint, credential);

var events = new List<EventGridEvent> {
    new EventGridEvent(
        subject: "orders/1234",
        eventType: "OrderPlaced",
        dataVersion: "1.0",
        data: new { OrderId = 1234, Total = 99.99 })
};

await client.SendEventsAsync(events);
```

Key Points

- Each event is up to **1 MB**
- Use `subject` for fine-grained filtering in subscriptions
- Event Grid retries delivery with exponential back-off for **24 hours**
- Dead-lettering requires a Blob Storage container configured on the subscription

📌 For high-volume publishing, batch events in a single `SendEventsAsync` call.

Day 4 Labs

Hands-on labs reinforce today's concepts. Each lab maps directly to a service or pattern covered in this session.

1

Azure Service Bus Messaging

Send and receive messages using queues with the .NET SDK

2

Publish/Subscribe with Topics

Create topics and filtered subscriptions for fan-out delivery

3

Event Grid Integration

Route Azure Blob Storage events to an Azure Function via Event Grid

4

Build Event-Driven Architecture

Combine Event Grid, Service Bus, and Functions into a cohesive async pipeline

Day 4 Key Takeaways



Message vs. Event

Messages transfer ownership and expect action. Events are notifications — the publisher doesn't own the outcome.



Service Bus for Enterprise Workflows

Choose Service Bus when you need ordering (sessions), transactions, pub/sub, dead-lettering, or duplicate detection.



Event Grid for Reactive Automation

Purpose-built for routing discrete events from Azure services or custom sources to serverless handlers with zero polling.



Event Hubs for Scale

When volume reaches millions of events per second — telemetry, IoT, clickstreams — Event Hubs is the right tool.